Assinador Digital de Documentos UTFPR

Documentação Técnica — Sprint #1
Explicação detalhada da arquitetura e de cada arquivo do projeto

Java 21 · Apache Maven · Spring Boot 4.0.8 · Apache PDFBox 3.0.8 · Bouncy Castle · PDF.js

Projeto de Extensão: Assinador Digital de Documentos — UTFPR
2026

1. Introdução e Objetivo

Este documento descreve, em detalhes, a implementação da Sprint #1 do Assinador Digital de Documentos da UTFPR. O objetivo do sistema é permitir que um usuário envie um arquivo PDF, posicione visualmente uma etiqueta de assinatura sobre o documento e receba de volta um PDF assinado digitalmente de forma criptográfica real — não apenas com uma imagem ou texto sobreposto.

A assinatura utiliza o padrão CMS/PKCS#7, incorporado ao PDF de forma compatível com leitores como o Adobe Acrobat Reader, permitindo verificar a autenticidade do documento e detectar qualquer alteração feita após a assinatura.

O documento a seguir explica, arquivo por arquivo, o papel de cada classe no sistema, o fluxo completo de uma requisição de assinatura, as decisões de segurança tomadas e as limitações atuais do projeto.

2. Tecnologias Utilizadas

- **Java 21** — versão exigida pelo projeto, LTS mais recente compatível com o Spring Boot 4.
- **Apache Maven** — gerenciador de dependências e build do projeto (`pom.xml`).
- **Spring Boot 4.0.8** (Spring Framework 7) — framework web, injeção de dependência, tratamento de upload multipart.
- **Apache PDFBox 3.0.8** — leitura, validação, manipulação e assinatura de PDFs (licença Apache 2.0).
- **Bouncy Castle** (`bcprov-jdk18on` / `bcpkix-jdk18on` 1.86) — geração da assinatura criptográfica CMS/PKCS#7 (licença Bouncy Castle, permissiva).
- **HTML / CSS / JavaScript puro** — frontend servido como recurso estático do próprio Spring Boot, sem framework de frontend.
- **PDF.js** (`pdfjs-dist` 6.3.289) — renderiza a prévia da página no navegador para o usuário posicionar a etiqueta antes de assinar; vendorizado localmente, sem depender de CDN em tempo de execução.

3. Arquitetura Geral

O projeto segue uma arquitetura em camadas clássica, separando responsabilidades:

```
Controller
    (recebe a requisicao HTTP)
        |
Service
    (valida, orquestra, decide regras de negocio)
        |
PDF / Signature Service
    (manipula o PDF e gera a assinatura criptografica)
        |
Bibliotecas (PDFBox + Bouncy Castle)
```

Fluxo completo de uma assinatura:

```
Usuario
  -> index.html (pagina "Assinar PDF")
  -> POST /api/assinaturas  (multipart/form-data)
  -> AssinaturaController        (recebe o MultipartFile)
  -> AssinaturaService           (valida upload, resolve posicao/nome)
  -> PdfService.validarConteudo  (confere se e um PDF real)
  -> CertificadoService          (carrega chave privada + certificado)
  -> PdfService.assinar          (monta etiqueta visual + campo de assinatura)
  -> PdfSignatureService         (gera a assinatura CMS/PKCS#7 via Bouncy Castle)
  -> PDF assinado retornado (application/pdf, Content-Disposition: attachment)
```

4. Estrutura de Pastas

```
src/main/java/br/edu/utfpr/td/tsi/assinadordigital/
+-- Main.java
+-- controller/
|   `-- AssinaturaController.java
+-- service/
|   +-- AssinaturaService.java
|   +-- PdfService.java
|   +-- CertificadoService.java
|   +-- PdfSignatureService.java
|   +-- MaterialAssinatura.java
|   +-- ResultadoAssinatura.java
|   +-- PosicaoAssinatura.java
|   `-- PosicaoAssinaturaRequest.java
+-- dto/
|   `-- ErroResponse.java
+-- config/
|   +-- AssinaturaKeystoreProperties.java
|   +-- PosicaoAssinaturaProperties.java
|   `-- UploadProperties.java
`-- exception/
    +-- ArquivoInvalidoException.java
    +-- PdfInvalidoException.java
    +-- CertificadoException.java
    +-- AssinaturaDigitalException.java
    `-- GlobalExceptionHandler.java

src/main/resources/
+-- application.properties
+-- certs/dev-keystore.p12
`-- static/
    +-- index.html
    +-- css/assinar.css
    +-- js/assinar.js
    `-- vendor/pdfjs/
```

5. Explicação de Cada Arquivo

5.1 Camada de Inicialização

`Main.java`

Ponto de entrada da aplicação. A anotação `@SpringBootApplication` diz ao Spring para escanear todo o pacote em busca de componentes (`@Service`, `@RestController`, `@Component`) e montá-los automaticamente. O método `main` apenas chama `SpringApplication.run(...)`, que sobe o servidor web embutido (Tomcat) e deixa a aplicação pronta para receber requisições.

```
@SpringBootApplication
public class Main {

    public static void main(String[] args) {
        SpringApplication.run(Main.class, args);
    }
}
```

5.2 Camada de Controller

`controller/AssinaturaController.java`

É a única "porta de entrada" HTTP do sistema. Define o endpoint `POST /api/assinaturas`, que recebe o arquivo (`MultipartFile`) e parâmetros opcionais: nome do assinante e a posição da etiqueta escolhida pelo usuário no preview (página/x/y/largura/altura). O controller não contém regra de negócio: ele só recebe a requisição, delega tudo para o `AssinaturaService` e monta a resposta HTTP (o PDF assinado, com cabeçalhos `Content-Type: application/pdf` e `Content-Disposition: attachment`).

```
@PostMapping(value = "/api/assinaturas", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public ResponseEntity<byte[]> assinar(
        @RequestParam("arquivo") MultipartFile arquivo,
        @RequestParam(value = "nomeAssinante", required = false) String nomeAssinante,
        @RequestParam(value = "pagina", required = false) Integer pagina,
        @RequestParam(value = "x", required = false) Float x,
        @RequestParam(value = "y", required = false) Float y,
        @RequestParam(value = "width", required = false) Float width,
        @RequestParam(value = "height", required = false) Float height) {

    PosicaoAssinaturaRequest posicaoRequest = new PosicaoAssinaturaRequest(pagina, x, y, width, height);
    ResultadoAssinatura resultado = assinaturaService.assinar(arquivo, nomeAssinante, posicaoRequest);

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_PDF);
    headers.setContentDisposition(
            ContentDisposition.attachment().filename(resultado.nomeArquivo(), StandardCharsets.UTF_8).build());

    return ResponseEntity.ok().headers(headers).body(resultado.conteudo());
}
```

5.3 Camada de Serviço — Orquestração

`service/AssinaturaService.java`

É o "maestro" do processo. Não sabe como assinar um PDF nem como ler um certificado — apenas organiza a ordem correta das etapas e delega para quem sabe fazer cada parte. O método principal, `assinar(...)`, executa nesta ordem:

- Valida o arquivo enviado (existe, não está vazio, extensão/Content-Type de PDF, tamanho máximo).
- Lê os bytes do arquivo e pede ao `PdfService` para confirmar que é um PDF válido de verdade (não só pela extensão).

- Pede ao `CertificadoService` a chave privada e o certificado.
- Decide o nome do assinante (o que o usuário digitou, ou o nome extraído do próprio certificado).
- Decide a posição final da etiqueta (a que o usuário escolheu no preview, ou a posição padrão do servidor).
- Chama o `PdfService` para assinar de fato, e devolve o resultado.

```
public ResultadoAssinatura assinar(MultipartFile arquivo, String nomeAssinante,
        PosicaoAssinaturaRequest posicaoRequest) {
    validarArquivo(arquivo);

    byte[] conteudoOriginal = lerConteudo(arquivo);
    pdfService.validarConteudo(conteudoOriginal);

    MaterialAssinatura material = certificadoService.carregarMaterialAssinatura();
    String assinante = StringUtils.hasText(nomeAssinante) ? nomeAssinante : nomeDoCertificado(material);
    PosicaoAssinatura posicao = resolverPosicao(posicaoRequest);

    byte[] conteudoAssinado = pdfService.assinar(conteudoOriginal, assinante, material, posicao);

    String nomeArquivoSaida = gerarNomeArquivoSaida(nomeSanitizado(arquivo.getOriginalFilename()));
    return new ResultadoAssinatura(conteudoAssinado, nomeArquivoSaida);
}
```

Um detalhe relevante: o nome do assinante, quando não informado pelo usuário, é extraído do certificado usando a classe padrão do Java `LdapName`, que separa o "Subject DN" do certificado (ex: `CN=Fulano,OU=TI,O=UTFPR`) e pega apenas o valor do atributo `CN` (nome comum) — em vez de mostrar a string inteira, ilegível, na etiqueta visual.

5.4 Camada de Serviço — Manipulação do PDF

`service/PdfService.java`

Responsável por tudo que envolve o documento PDF em si: validar o conteúdo recebido, desenhar a etiqueta visual e montar o campo de assinatura na posição correta. Tem duas responsabilidades principais:

a) Validar o conteúdo (não confiar só na extensão):

```
public void validarConteudo(byte[] conteudo) {
    if (!iniciaComAssinaturaPdf(conteudo)) {
        throw new ArquivoInvalidoException("Formato de arquivo invalido. Envie um PDF.");
    }
    try (PDDocument documento = Loader.loadPDF(conteudo)) {
        if (documento.getNumberOfPages() == 0) {
            throw new PdfInvalidoException("O arquivo PDF nao pode ser processado.");
        }
    } catch (IOException e) {
        throw new PdfInvalidoException("O arquivo PDF nao pode ser processado.", e);
    }
}
```

Primeiro confere os 5 bytes mágicos (`%PDF-`) no início do arquivo; só depois tenta efetivamente abrir o documento com o PDFBox — se a estrutura interna estiver corrompida, o PDFBox lança `IOException`, que é convertida em `PdfInvalidoException` (HTTP 422).

b) Assinar (metadados + posicionamento + representação visual):

```
PDSignature assinatura = new PDSignature();
assinatura.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
assinatura.setSubFilter(PDSignature.SUBFILTER_ADBE_PKCS7_DETACHED);
assinatura.setName(nomeAssinante);
assinatura.setReason("Assinatura digital via Assinador Digital UTFPR");
assinatura.setSignDate(Calendar.getInstance());

// PDVisibleSignDesigner.yAxis() conta a partir do TOPO da pagina,
// diferente da convencao do PDF (origem embaixo) -- e preciso converter:
float yAPartirDoTopo = pagina.getMediaBox().getHeight() - posicao.y() - posicao.height();

PDVisibleSignDesigner designer = new PDVisibleSignDesigner(documento,
        new ByteArrayInputStream(imagemEtiqueta), indicePagina + 1);
designer.xAxis(posicao.x()).yAxis(yAPartirDoTopo)
        .width(posicao.width()).height(posicao.height())
        .adjustForRotation();

documento.addSignature(assinatura, new PdfSignatureService(material), opcoes);
documento.saveIncremental(saida);
```

Os campos `Filter`/`SubFilter` são os metadados que fazem qualquer leitor de PDF reconhecer aquilo como uma assinatura digital padrão (Adobe.PPKLite / adbe.pkcs7.detached). A conversão do eixo Y foi um bug real descoberto durante os testes: a biblioteca do PDFBox usada para posicionar a etiqueta (`PDVisibleSignDesigner`) conta a coordenada verticalmente a partir do topo da página, e não de baixo como é a convenção normal de PDF — sem essa correção, a etiqueta aparecia sempre perto do topo do documento, mesmo configurada para o canto inferior.

A etiqueta visual em si é desenhada em memória como uma imagem PNG (método `gerarImagemEtiqueta`), usando `java.awt.Graphics2D`: um retângulo arredondado com fundo claro e borda azul, contendo o texto "DOCUMENTO ASSINADO DIGITALMENTE", o nome do assinante e a data/hora. O tamanho da fonte é reduzido automaticamente (método `desenharLinha`) quando o texto é maior do que cabe na caixa, evitando que nomes longos estourem visualmente a etiqueta.

Também existe um ajuste de segurança (`ajustarAosLimitesDaPagina`) que recalcula x/y/largura/altura para garantir que a etiqueta sempre fique dentro dos limites reais da página, mesmo que o cliente (navegador) envie coordenadas inválidas.

5.5 Camada de Serviço — Assinatura Criptográfica

`service/PdfSignatureService.java`

Esta é a classe que faz a parte puramente criptográfica: recebe o intervalo de bytes do PDF (calculado internamente pelo PDFBox) e devolve uma assinatura CMS/PKCS#7 detached, usando a chave privada e a cadeia de certificados carregadas pelo `CertificadoService`. Implementa a interface `SignatureInterface` do PDFBox.

```
@Override
public byte[] sign(InputStream content) throws IOException {
    X509Certificate certificado = (X509Certificate) material.cadeiaCertificados()[0];

    ContentSigner contentSigner = new JcaContentSignerBuilder("SHA256withRSA")
            .setProvider(BouncyCastleProvider.PROVIDER_NAME)
            .build(material.chavePrivada());

    CMSSignedDataGenerator gerador = new CMSSignedDataGenerator();
    gerador.addSignerInfoGenerator(new JcaSignerInfoGeneratorBuilder(
            new JcaDigestCalculatorProviderBuilder().setProvider(BouncyCastleProvider.PROVIDER_NAME).build())
            .build(contentSigner, certificado));
    gerador.addCertificates(new JcaCertStore(Arrays.asList(material.cadeiaCertificados())));

    byte[] conteudo = content.readAllBytes();
    CMSSignedData signedData = gerador.generate(new CMSProcessableByteArray(conteudo), false);
    return signedData.getEncoded();
}
```

Em resumo: assina os bytes do documento com SHA-256 + RSA, usando a biblioteca Bouncy Castle, e monta um pacote CMS contendo a assinatura e o certificado do assinante — é esse pacote que é embutido no PDF e reconhecido por qualquer leitor compatível como uma assinatura digital real.

5.6 Camada de Serviço — Certificado Digital

`service/CertificadoService.java`

Responsável por carregar o certificado digital (arquivo `.p12`) e validar cada etapa: existência do arquivo, senha correta, alias existente, validade do certificado (não expirado). Isolar essa lógica numa classe própria permite que, no futuro, o keystore em arquivo seja substituído por outro mecanismo (ICP-Brasil, HSM, serviço de nuvem) sem impactar o resto do sistema.

- Arquivo do certificado inexistente → `CertificadoException` ("Certificado de assinatura não encontrado.")
- Senha incorreta → `CertificadoException` ("Senha do certificado de assinatura incorreta.")
- Alias inexistente → `CertificadoException` ("Alias do certificado de assinatura não encontrado.")
- Certificado expirado → `CertificadoException` ("O certificado de assinatura está expirado.")

O caminho, a senha, o alias e o tipo do keystore vêm de `AssinaturaKeystoreProperties`, que por sua vez lê variáveis de ambiente (`SIGNATURE_KEYSTORE_PATH`, etc.) — a senha nunca está escrita diretamente no código-fonte.

5.7 Objetos de Dados Auxiliares (records)

Quatro classes pequenas, do tipo `record` do Java (imutáveis, sem lógica além de carregar dados), usadas para passar informação entre as camadas de serviço:

- `MaterialAssinatura` — chave privada + cadeia de certificados, devolvido pelo `CertificadoService`.
- `ResultadoAssinatura` — bytes do PDF assinado + nome do arquivo final, devolvido pelo `AssinaturaService` ao controller.
- `PosicaoAssinatura` — a posição final já decidida (página/x/y/largura/altura) que será usada de fato.

- `PosicaoAssinaturaRequest` — a posição que *pode* ter vindo do formulário (campos anuláveis, já que são opcionais).

5.8 Configuração (application.properties)

Três classes `@ConfigurationProperties`, cada uma representando um grupo de configurações do `application.properties`:

- **AssinaturaKeystoreProperties** (`signature.keystore.*`) — caminho, senha, alias e tipo do certificado.
- **PosicaoAssinaturaProperties** (`assinatura.posicao.*`) — posição/tamanho padrão da etiqueta (página, x, y, largura, altura), centralizando esses valores num único lugar em vez de espalhados pelo código.
- **UploadProperties** (`assinatura.upload.*`) — tamanho máximo de upload aceito (camada extra de validação, além do limite do próprio Spring).

Conteúdo completo do `application.properties`:

```
spring.application.name=assinadordigital
server.port=${SERVER_PORT:8080}

spring.servlet.multipart.max-file-size=${ASSINATURA_UPLOAD_MAX_SIZE:10MB}
spring.servlet.multipart.max-request-size=${ASSINATURA_UPLOAD_MAX_SIZE:10MB}
assinatura.upload.max-size-bytes=${ASSINATURA_UPLOAD_MAX_SIZE_BYTES:10485760}

assinatura.posicao.pagina=${ASSINATURA_POSICAO_PAGINA:-1}
assinatura.posicao.x=${ASSINATURA_POSICAO_X:350}
assinatura.posicao.y=${ASSINATURA_POSICAO_Y:100}
assinatura.posicao.width=${ASSINATURA_POSICAO_WIDTH:200}
assinatura.posicao.height=${ASSINATURA_POSICAO_HEIGHT:80}

signature.keystore.path=${SIGNATURE_KEYSTORE_PATH:classpath:certs/dev-keystore.p12}
signature.keystore.password=${SIGNATURE_KEYSTORE_PASSWORD:changeit}
signature.keystore.alias=${SIGNATURE_KEYSTORE_ALIAS:assinador-dev}
signature.keystore.type=${SIGNATURE_KEYSTORE_TYPE:PKCS12}
```

5.9 Tratamento de Erros

Quatro exceções de negócio, cada uma representando um tipo de falha, mais uma classe central que as intercepta e transforma em respostas HTTP amigáveis:

- `ArquivoInvalidoException` → HTTP 400 (arquivo ausente, extensão errada, tamanho excedido).
- `PdfInvalidoException` → HTTP 422 (PDF corrompido, extensão certa mas conteúdo inválido).
- `CertificadoException` → HTTP 500 (problema no certificado configurado no servidor).
- `AssinaturaDigitalException` → HTTP 500 (falha ao gerar a assinatura).

`exception/GlobalExceptionHandler.java`

Anotada com `@RestControllerAdvice`, essa classe intercepta automaticamente qualquer exceção lançada em qualquer lugar do sistema (não precisa de `try/catch` espalhado pelo código) e devolve sempre um JSON no formato `ErroResponse`, nunca uma stack trace técnica para o usuário final:

```
@ExceptionHandler(PdfInvalidoException.class)
public ResponseEntity<ErroResponse> handlePdfInvalido(PdfInvalidoException ex) {
    log.warn("PDF nao processavel: {}", ex.getMessage());
    return ResponseEntity.status(HttpStatus.UNPROCESSABLE_CONTENT).body(ErroResponse.of(ex.getMessage()));
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ErroResponse> handleGeneric(Exception ex) {
    log.error("Erro inesperado ao processar assinatura", ex);
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(ErroResponse.of("Ocorreu um erro inesperado. Tente novamente mais tarde."));
}
```

Também trata exceções do próprio Spring, como `MissingServletRequestPartException` (upload sem o campo "arquivo") e `MaxUploadSizeExceededException` (arquivo maior que o limite configurado), convertendo ambas em respostas 400 amigáveis.

5.10 DTO de Resposta

`dto/ErroResponse.java`

Um `record` simples que define o formato padrão de qualquer resposta de erro da API:

```
public record ErroResponse(boolean sucesso, String mensagem) {
    public static ErroResponse of(String mensagem) {
        return new ErroResponse(false, mensagem);
    }
}
```

5.11 Frontend (Página "Assinar PDF")

`static/index.html + static/css/assinar.css`

A página exibe um cartão centralizado com título, texto explicativo, campo de seleção de arquivo (aceitando apenas `.pdf`) e o botão "Assinar", que fica desabilitado até um PDF válido ser escolhido. Depois de escolher o arquivo, aparece uma área de prévia com navegação entre páginas e uma caixa arrastável/redimensionável representando a etiqueta de assinatura.

`static/js/assinar.js`

Módulo JavaScript responsável por: validar o arquivo escolhido no navegador (extensão/tipo); carregar o PDF localmente com a biblioteca PDF.js e desenhar a página num `<canvas>`; permitir arrastar e redimensionar a etiqueta com eventos de ponteiro (`pointerdown/pointermove/pointerup`); converter a posição escolhida (em pixels de tela) para pontos PDF antes de enviar; montar o `FormData` e enviar via `fetch` para `POST /api/assinaturas`; e, ao receber a resposta, disparar o download

automático do PDF assinado. Se a prévia falhar por qualquer motivo (ex.: PDF protegido por senha), a assinatura segue normalmente usando a posição padrão do servidor.

`static/vendor/pdfjs/`

Biblioteca PDF.js (Mozilla, licença Apache 2.0) hospedada localmente dentro do próprio projeto, para renderizar a prévia do PDF no navegador sem depender de nenhum serviço externo em tempo de execução.

5.12 Certificado de Desenvolvimento

`certs/dev-keystore.p12`

Um keystore PKCS#12 autoassinado (alias `assinador-dev`, senha `changeit`), gerado com `keytool` e incluído no projeto apenas para desenvolvimento e testes. **Não possui o mesmo valor jurídico de um certificado ICP-Brasil** — leitores de PDF exibirão a assinatura como "identidade não verificada", o que é o comportamento esperado para um certificado de teste.

6. Segurança e Boas Práticas Aplicadas

- Senha e caminho do certificado vêm de variáveis de ambiente, nunca escritos diretamente no código-fonte.
- O conteúdo do arquivo é validado de verdade (assinatura mágica `%PDF-` + abertura real com PDFBox), não só a extensão `.pdf`.
- Limite de tamanho de upload aplicado em duas camadas: pelo Spring (`spring.servlet.multipart.max-file-size`) e por regra de negócio própria (`UploadProperties`).
- Nome de arquivo enviado pelo usuário é sanitizado antes de ser usado (remove caracteres inválidos/perigosos).
- Exceções técnicas nunca são expostas ao usuário final — toda resposta de erro passa pelo `GlobalExceptionHandler` e usa mensagens amigáveis.
- Logs registram eventos do fluxo (recebimento, início/fim da assinatura) mas nunca senha, chave privada ou conteúdo do PDF.
- Posição da etiqueta enviada pelo cliente é sempre validada/ajustada no servidor contra os limites reais da página, mesmo que o navegador envie valores inválidos.

7. Limitações Atuais e Próximos Passos

Limitações da Sprint #1:

- Não há autenticação de usuários: o nome do assinante é opcional (via formulário) ou extraído do certificado configurado no servidor.
- O certificado de desenvolvimento é autoassinado e não possui validade jurídica; produção exigiria um certificado real (ICP-Brasil ou equivalente).
- O editor visual de posicionamento funciona por página única de cada vez (não há múltiplas etiquetas na mesma assinatura).

Possíveis próximos passos:

- Suporte a múltiplos assinantes por documento.
- Integração com certificados ICP-Brasil reais (A1/A3, cartão/token).
- Histórico/consulta de documentos assinados (persistência em banco de dados).
- Autenticação de usuários e vínculo do documento à conta do usuário.
- Página de "Verificar Assinaturas" (inspirada no Assinador ICPEdu da RNP), permitindo enviar um PDF já assinado para conferir sua validade.
- Autenticação institucional federada (CAFe) e certificado pessoal por usuário, nos moldes do ICPEdu.

Documento gerado como material de apoio técnico ao projeto Assinador Digital de Documentos — UTFPR, Sprint #1.